IBM BOB HACKATHON | SDLC · JAVA MODERNISATION

Problem Statements

Five hands-on challenges for technical developers to solve using IBM Bob. Each problem is scoped to fit within the 3-hour window. A sample codebase is provided for every problem statement — clone the repo, open your assigned folder, and follow the README.
Deliverables are for demo and showcase

SDLC

01 Intelligent Pull Request Reviewer

Build a tool that automates code review and enforces quality gates before code reaches main

THE PROBLEM

> Your team merges dozens of pull requests daily. Code reviews are inconsistent — senior developers catch different things, junior developers miss security gaps, and no one has time to review everything thoroughly. Bugs and vulnerabilities slip into production because review quality depends entirely on who is available that day.

YOUR TASK

Step 1 Use Bob to design and build a CLI tool (Python, Node.js, or any language) that reads Java source files, sends them to an LLM with a structured review prompt, and outputs a severity-sorted finding report (Critical / High / Medium / Low).

Step 2 Use Bob to engineer the review system prompt — specify the output format (JSON array of findings with file, line, explanation, and fix), and instruct the model to look for injection vulnerabilities, hardcoded secrets, logic bugs, missing auth, and non-atomic operations.

Step 3 Run your tool against the sample payment transfer service provided in the repo. Iterate the prompt with Bob until the findings are specific, actionable, and correctly severity-ranked.

Step 4 Feed the most critical finding back to Bob and ask it to regenerate the corrected version of that file — show the full fixed file with an explanation of every change.

DELIVERABLE

Demo-ready output: A working CLI tool that produces a structured finding report when run against the sample codebase — at least 3 findings at different severity levels, one prompt iteration showing improvement, and a corrected file generated by Bob for the most critical issue.

Bob tip: Use **Agent mode**. Start with: "I want to build a Python CLI that reads Java files, sends them to an LLM, and outputs a severity-sorted code review report. Help me design the structure and write the LLM call." Then iterate the prompt until the tool catches everything.

SDLC

02 Requirement-to-Test Coverage Generator

Build a tool that turns a business requirement directly into a test suite skeleton

THE PROBLEM

> Business analysts write requirements. Developers build features. QA engineers write test cases — from scratch, manually, often late. The result is patchy test coverage, missed edge cases, and production defects that were entirely predictable from the original requirement. In regulated industries like banking and insurance, missing a boundary condition isn't just a bug — it can be a compliance failure.

YOUR TASK

Step 1 Use Bob to design and build a CLI tool that accepts a plain-English business requirement as input, sends it to an LLM, and outputs a JUnit 5 test suite skeleton — with descriptive method names, inline comments, and TODO bodies ready for implementation.

Step 2 Use Bob to engineer the extraction prompt — instruct the model to group testable conditions into four categories: happy path, negative cases, boundary values, and compliance / edge cases. Output must be valid Java with a minimum of 8 test methods.

Step 3 Run your tool against the four banking requirements provided in the repo (transfer limits, interest calculation, international fees, fraud flagging). Iterate the prompt with Bob until boundary values and compliance cases are covered.

Step 4 Extend the tool with a second LLM call that identifies ambiguities in the requirement — things that are untestable as written — and suggests how to rewrite them to be precise.

DELIVERABLE

Demo-ready output: A working CLI tool that takes a banking requirement as input and produces a JUnit 5 test skeleton — at least 8 test methods covering 3+ categories, one prompt iteration showing improved boundary coverage, and an ambiguity report for one of the requirements.

Bob tip: Use **Agent mode**. Start with: "I want to build a Python CLI that takes a business requirement as input and generates a JUnit 5 test suite skeleton. Help me design the tool and write the system prompt that extracts testable conditions." Then test it against real banking requirements and refine.

SDLC

03 Legacy Codebase Onboarding Accelerator

Build an interactive agent that gets a new developer productive on an unfamiliar codebase in minutes

THE PROBLEM

> Every organisation has that codebase — 10 years old, no documentation, original developers long gone. A new developer joining the team spends their first two weeks just trying to understand what the system does and where to make a change safely. In financial institutions, this onboarding risk is amplified because touching the wrong class can have downstream effects on calculations, reporting, or regulatory outputs. Static documentation goes stale. What a new developer actually needs is someone they can ask.

YOUR TASK

Step 1 Use Bob to design and build a conversational CLI agent that loads all source files from a directory at startup and runs an interactive question-answer loop — the developer types plain-English questions, the agent responds with accurate, code-grounded answers.

Step 2 Use Bob to engineer the system prompt — the agent should behave like a senior engineer who knows the codebase, flag risky areas, suggest safe entry points, and never fabricate functionality that isn't in the code.

Step 3 Test your agent against the deliberately obfuscated legacy loan engine provided in the repo — cryptic method names, magic numbers, shared mutable static state, no comments. Ask it the benchmark questions in the README and iterate until answers are accurate and specific.

Step 4 Use Bob to add conversation memory — a sliding window or summary mechanism so the agent remembers earlier questions. Add special commands: `/risk` (produce a risk map) and `/readme` (generate module documentation).

DELIVERABLE

Demo-ready output: A working conversational CLI agent that loads the legacy codebase and answers developer questions live — demonstrate at least 3 benchmark questions answered correctly, a follow-up that uses conversation memory, and one improvement iteration made with Bob.

Bob tip: Use **Agent mode**. Start with: "I want to build a conversational CLI agent that loads a Java codebase at startup and answers developer questions about it. Help me design the architecture and write the system prompt." The harder the codebase, the more impressive the demo.

JAVA MODERNISATION

04 Java 8 → Java 21 Migration Planner & Refactorer

Upgrade a real Java 8 service to idiomatic Java 21 with a clear, phased plan

THE PROBLEM

> Java 8 reached end-of-life for free public updates years ago, yet the majority of enterprise Java applications in banking, insurance, and capital markets still run on it. Upgrading is not just about changing a version number — Java 9 through 21 introduced module system changes, removed APIs, deprecated patterns, and added powerful new language features. Without a structured approach, teams either stall or break production upgrading blindly.

YOUR TASK

Step 1 A Java 8 customer account service is provided in the repo. It uses legacy patterns across date/time handling, type safety, concurrency, resource management, and class design. Open it in Bob (Java Modernisation mode) and ask for a full migration assessment.

Step 2 Use Bob (Java Modernisation mode) to analyse the code and produce a migration assessment: every legacy pattern that should be modernised, the Java 21 feature that replaces it, and why the modern approach is better.

Step 3 Ask Bob to generate the fully refactored Java 21 version of each file — applying idiomatic modern Java throughout. Generate each file in full, not just the changed parts.

Step 4 Ask Bob to produce an annotated before/after comparison for the 5 most impactful changes — Java 8 snippet, Java 21 snippet, and a plain-English explanation of why it's better, as if presenting at a team tech talk.

DELIVERABLE

Demo-ready output: A migration assessment report, a fully refactored Java 21 version of at least one file, and an annotated before/after comparison for at least 2 changes — all generated through Bob in Java Modernisation mode, presented as a team knowledge-share.

Bob tip: Switch Bob to **Java Modernisation mode** before pasting your code. Ask it to "produce a before/after comparison with a plain-English annotation for each change, as if preparing a team tech talk."

JAVA MODERNISATION

05 WebSphere to Open Liberty Re-platforming Guide

Produce a concrete, code-level migration plan for moving a real application off WebSphere

THE PROBLEM

Traditional WebSphere Application Server (WAS) is expensive to license, slow to deploy, and incompatible with modern cloud-native delivery practices. Financial institutions that still run WAS face mounting pressure to re-platform — but the migration is non-trivial. WebSphere-specific APIs, proprietary deployment descriptors, EJB patterns, custom classloader configurations, and transaction manager bindings all create migration blockers that teams don't discover until they try to deploy on Liberty and things break.

YOUR TASK

Step 1 A WebSphere 8.5 trade settlement service is provided in the repo — an EJB session bean, proprietary WAS binding files (`ibm-ejb-jar-bnd.xml`, `ibm-web-bnd.xml`), WAS-specific Java APIs, and an admin-console datasource config. Open it in Bob (Java Modernisation mode).

Step 2 Use Bob (Java Modernisation mode) to identify every WAS-specific construct in the code and configuration that will not work on Open Liberty out of the box — with a clear explanation of why each one is a blocker.

Step 3 Ask Bob to produce the Liberty-compatible replacement for each blocker — including updated Java code, the correct `server.xml` feature configuration, and any Jakarta EE 10 API changes required.

Step 4 Ask Bob to produce a migration checklist ordered by effort (low to high) that a team could use as a sprint-by-sprint re-platforming backlog.

DELIVERABLE

Demo-ready output: A blocker inventory with code-level fixes, a corrected `server.xml` snippet, and a prioritised migration checklist — all produced through Bob in Java Modernisation mode, demonstrating a real path from WAS to Liberty the team could act on immediately.

Bob tip: Use **Java Modernisation mode**. Ask Bob to "act as a WebSphere-to-Liberty migration architect and produce a technical migration guide I can share with my development team and present to my architect."